Test Plan: Swag Labs Web Shop (saucedemo.com)

Document ID	TP-SL-001
Version	1.0
Author	Davit Jikurashvili, Senior QA Engineer
Date	17 October 2026
Application under test	Swag Labs, https://www.saucedemo.com (public demo e-commerce app)
Status	Final (see section 12, Approvals)

---

1. Purpose

This plan defines the scope, approach, resources, schedule and exit criteria for functional testing of the Swag Labs web shop. It covers the core purchase journey: logging in, browsing and sorting products, managing the cart and completing checkout.

2. Application overview

Swag Labs is a single-page web shop. A user logs in, sees a catalogue of 6 products, can sort them, add or remove them from the cart, view product details and buy them through a 3-step checkout (customer information → order overview → confirmation).

The app ships with several test accounts. Each one simulates a different kind of defect, so the same test cases can be run against each account to show how failures appear:

Account	Intended behaviour
`standard_user`	Baseline: the app should work correctly
`locked_out_user`	Login must be refused
`problem_user`	UI and functional defects
`performance_glitch_user`	Slow responses
`error_user`	Errors in cart and checkout actions
`visual_user`	Visual and layout defects

Test item: the production demo deployed at https://www.saucedemo.com on 17 October 2026. The site exposes no version or build number, so the test date identifies the build.

3. Scope

3.1 In scope

Area	Features
Authentication	Login, field validation, locked account, logout, access control on protected pages
Product catalogue	Product list, images, prices, product detail page, sorting (A→Z, Z→A, price ↑, price ↓)
Cart	Add / remove from the list and from the detail page, cart badge, cart page, cart persistence
Checkout	Customer information form and validation, order overview and price calculation (item total + tax = total), order completion
Cross-browser	Desktop Chromium and Firefox (the builds bundled with Playwright 1.64)
Non-functional (light)	Basic page responsiveness; obvious visual and layout defects

3.2 Out of scope

- Real payments and shipping (the app uses fixed demo payment data)
- Back-end and database testing (no access to the server side)
- Load, stress and security penetration testing
- Native mobile apps and mobile browsers
- Third-party links in the footer (Twitter, Facebook, LinkedIn)

4. Test approach

Level / type	Approach
Smoke	5 `@smoke` -tagged automated checks (3 UI: login, add/remove, end-to-end purchase; 2 API: health, create booking). A failing smoke check fails the CI run.
Functional	Scripted test cases (see *02-Test-Cases*) designed with equivalence partitioning (valid / invalid / empty / locked credentials) and state transitions (cart states, checkout steps).
Negative	Invalid and empty input, locked account, direct URL access without login.
Regression	The automated Playwright suite (UI + API) runs in CI on every change.
Exploratory	Time-boxed sessions with charters (e.g. "explore checkout with unusual names and long input") when time allows. Findings are logged as defects or new test cases.
Cross-browser	The 18 automated cases run in both Chromium and Firefox; the other 17 cases are executed in Chromium only.
Account-based (persona) testing	11 key cases repeated for every demo account to show account-specific defects (`scripts/run-account-matrix.js`).

Automation

Repeatable, high-value cases (18 of 35) are automated with **Playwright + TypeScript** (Page Object Model) and run in **GitHub Actions**: https://github.com/da96vit/playwright-qa-portfolio. Each automated test is named after the test case IDs it covers. The other 17 cases are not part of the regression suite. In this cycle they are executed with a one-off Playwright script (`scripts/execute-manual-cases.js`), so every result can be re-run and checked.

5. Entry and exit criteria

Entry criteria

- The test environment (saucedemo.com) is reachable
- Test accounts are available and the password is known
- Test cases are reviewed and approved

Exit criteria

- 100% of planned test cases executed
- 100% of smoke and *Critical* priority cases pass for `standard_user`
- No open *Critical* or *High* severity defects on the baseline account (`standard_user`)
- All defects found are logged with steps, evidence and severity
- The test summary report is delivered

Suspension criteria

- The environment is down for more than 1 hour, or login fails for `standard_user` (testing is blocked)

Resumption criteria

- The environment is reachable again and `standard_user` can log in; the smoke checks are re-run before execution continues

6. Test environment

Item	Value
URL	https://www.saucedemo.com
Browsers	Playwright 1.64 bundled builds: Chromium 156 and Firefox 157 (desktop). Viewport 1280×720 in the automated suite, 1280×800 in the execution and evidence scripts
OS	Windows 11; Ubuntu (CI)
Test data	Demo accounts listed above, password `secret_sauce`; customer data: Davit / Tester / 0179; expected product names and prices as listed in TC-012
Tools	Playwright (automation and scripted execution), GitHub Actions (CI), Markdown bug reports with Jira-style fields (no tracker is connected to this public demo)

7. Defect management

Each defect is logged with: ID, title, environment, account, preconditions, steps to reproduce, expected vs actual result, severity, priority and evidence (screenshot, console log).

Severity	Definition
Critical	Blocks the main purchase flow with no workaround, or causes financial or legal exposure (e.g. the price shown differs from the price charged)
High	Major function broken or wrong data shown; workaround is hard
Medium	Function partly broken; a reasonable workaround exists

Severity	Definition
Low	Cosmetic issue or minor inconvenience

Defect life cycle: *New* → *Open* → *In Progress* → *Fixed* → *Retest* → *Closed* (or *Reopened*).

8. Risks and mitigation

Risk	Impact	Mitigation
Public demo site changes without notice	Test cases or automation break	Stable `data-test` locators; review failures before logging defects
Environment slow or unavailable	Schedule slips	Retry policy in CI; run critical cases first
Demo defects mask real regressions	Missed bugs	Use `standard_user` as the baseline; compare other accounts against it

9. Deliverables

1. Test Plan (this document)
2. Test Cases with a traceability matrix (*02-Test-Cases*)
3. Bug Reports (*03-Bug-Reports*)
4. Test Summary Report (*04-Test-Summary-Report*)
5. Automated test suite and CI pipeline (GitHub)

10. Schedule (example 1-week cycle)

Day	Activity
1	Requirements review, test plan, test case design
2	Test case review; test data setup; automation smoke suite
3–4	Test execution (all accounts; automated cases in both browsers); exploratory sessions; defect logging
5	Retesting, regression run, Test Summary Report

11. Roles

Role	Responsibility
QA Engineer (Davit Jikurashvili)	Test planning, design, execution, automation, reporting
Developer	Defect fixing, unit tests
Product Owner	Requirement clarification, release decision

12. Approvals

Role	Name	Decision
QA Engineer (author)	Davit Jikurashvili	Approved for execution, 17 October 2026

Role	Name	Decision
Product Owner	Not applicable: saucedemo.com is a public demo without a product team	